🛡️ Conflict Escalation Early-Warning System

System Architecture & Design Document

🏗️ SECTION 1: SYSTEM ARCHITECTURE & DATA FLOW

1.1 Architecture Pattern

Chosen Architecture: Modular Monolith

Decision

The system is implemented as a **Modular Monolithic architecture**, where all components run as a single application but are logically separated into modules.

Justification

- Faster development for a student project
- Easier debugging and testing
- No DevOps overhead (service discovery, orchestration)
- Clean code separation without deployment complexity

Internal Modules

```
/ingestion        → API data collection
/storage          → Database access
/features         → Feature engineering
/model            → ML training & inference
/api              → Backend endpoints
```

```
/frontend          → Dashboard & visualization
```

1.2 High-Level Data Flow (Confirmed)

Final Data Path:

```
External APIs
   ↓
Task Scheduler (Cron)
   ↓
Asynchronous Ingestion Layer
   ↓
PostgreSQL Database (Raw Data)
   ↓
Feature Engineering
   ↓
Feature Store (PostgreSQL)
   ↓
ML Inference
   ↓
Results Store
   ↓
Frontend Dashboard
```

Note on Vector Databases

A **vector database is NOT used** because:

- The project uses **feature-based ML**, not semantic search
- No large-scale embedding retrieval is required
- Adds unnecessary complexity for this scope

1.3 Concurrency & API Handling

Approach: Asynchronous Programming

- Python `asyncio` with `aiohttp` / `httpx`
- Multiple APIs are fetched **in parallel**
- Prevents blocking and reduces latency

Task Scheduling

- **Cron jobs** trigger ingestion every 30–60 minutes
- Celery is not required at this scale

💾 SECTION 2: DATA PERSISTENCE (SYSTEM MEMORY)

2.1 Database Choice

Chosen Database: PostgreSQL + PostGIS

Justification

- Structured relational data
- Native geographic support
- Time-series friendly
- Industry standard and viva-friendly

Why not MongoDB?

- Weak relational joins
- Poor temporal aggregation
- Not ideal for analytics-heavy workloads

2.2 Schema Design

Core Linking Strategy: Region_Key

All data sources are linked using a common **Region_Key** rather than raw coordinates or event IDs.

Tables Overview

regions

```
region_key (PK)
region_name
country
geometry (PostGIS)
```

news_articles

```
article_id (PK)
region_key (FK)
published_time
headline
article_text
source
```

conflict_events

```
event_id (PK)
region_key (FK)
event_type
event_time
severity
```

features

```
feature_id (PK)
region_key (FK)
time_window
```

```
event_acceleration
sentiment_volatility
protest_ratio
```

Why Region_Key?

- Clean aggregation
- Avoids noisy event-to-event joins
- Matches analyst workflows

2.3 Caching Strategy

Chosen Strategy: Hybrid Cache

1. **Database Cache**
    a. API responses stored with timestamps
    b. API is queried only if cached data is stale
2. **Local In-Memory Cache**
    a. Python dictionary / LRU cache
    b. Short TTL (5–15 minutes)

Why not Redis?

- Adds operational complexity
- Not required for single-user or low-traffic systems

🧠 SECTION 3: ML / AI PIPELINE

3.1 Feature Store

Storage: PostgreSQL (Feature Table)

Features Stored

- Event frequency
- Event acceleration
- Sentiment volatility
- Protest-to-violence ratio
- Diplomatic silence duration

Each feature is stored per:

- Region
- Time window (e.g., last 7 days)

3.2 Model Serving Strategy

Model Loading: In-Memory at Server Startup

Justification

- Low latency inference
- Simple deployment
- No inter-service communication

Separate ML microservices are unnecessary for a single model.

3.3 Inference Frequency

Chosen Strategy: Hybrid

Mode	Purpose
Scheduled inference	Hourly baseline updates
On-demand inference	User-triggered analysis

This balances performance and responsiveness.

💻 SECTION 4: FRONTEND & VISUALIZATION BRIDGE

4.1 Frontend State Management

Chosen Method: Polling

- Frontend requests updates every 60 seconds
- Simple and reliable
- No WebSocket complexity

WebSockets are avoided as real-time alerts are not critical.

4.2 Map Engine Logic

Rendering: Client-Side

Tools

- MapLibre / Plotly Mapbox

Justification

- Interactive heatmaps
- Zoom & pan support
- Lower server load

Only **aggregated geographic data** is sent to the frontend.

🛡️ SECTION 5: DEVSECOPS & DEPLOYMENT

5.1 Containerization

Docker: Yes

- Ensures environment consistency
- Simplifies demo and deployment
- Professional best practice

Single container deployment is sufficient.

5.2 Secrets Management

Method: `.env` + Environment Variables

- API keys stored in `.env`
- `.env` added to `.gitignore`
- Accessed using `os.environ`

Prevents accidental key leakage on GitHub.

5.3 CI/CD

Status: Optional but Recommended

- GitHub Actions for:
  - Linting
  - Unit tests
  - Build verification

Not mandatory, but adds project polish.

📌 FINAL ARCHITECTURE SUMMARY

The system is implemented as a modular monolith using Python, with asynchronous data ingestion, PostgreSQL with PostGIS for spatial and temporal storage, and a scheduled machine learning inference pipeline. Feature engineering and model inference are tightly integrated, and results are served through a lightweight Python-based dashboard. The architecture prioritizes simplicity, explainability, and scalability within a student-friendly scope.